

The Knuth–Morris–Pratt (KMP) Algorithm

1 Introduction

Main Idea

The Knuth–Morris–Pratt (KMP) algorithm is a linear-time string-matching algorithm. It avoids unnecessary re-comparisons by using information about how the pattern matches against itself.

Suppose:

$$T[1 : n] = \text{text}, \quad P[1 : m] = \text{pattern}.$$

The naive algorithm may compare many characters repeatedly. KMP improves this by precomputing an auxiliary array:

$$\pi[1 : m],$$

called the **prefix function**.

Main Benefit

KMP matches the pattern in the text in:

$$\Theta(n)$$

time, after preprocessing the pattern in:

$$\Theta(m)$$

time.

So total running time is:

$$\Theta(n + m).$$

2 Why KMP is Needed

When the naive matcher has already matched several characters and then finds a mismatch, it usually shifts the pattern by only one position and starts comparing again from the beginning.

That causes repeated work.

Key Observation

If we already know that some prefix of the pattern matched the text, then after a mismatch we should not forget all that information.

Instead, we should determine the longest prefix of the pattern that could still be a suffix of the matched part.

That is exactly the information stored by the prefix function.

3 The Prefix Function

3.1 Definition

For a pattern $P[1 : m]$, the prefix function is an array:

$$\pi[1 : m].$$

Prefix Function

For each $q = 1, 2, \dots, m$,

$$\pi[q] = \max\{k \mid k < q \text{ and } P_k \text{ is a suffix of } P_q\}.$$

So

$$\pi[q]$$

is the length of the **longest proper prefix** of $P[1 : q]$ that is also a suffix of $P[1 : q]$.

3.2 Meaning

If a mismatch occurs after matching q characters, then $\pi[q]$ tells us the length of the best smaller prefix that can still continue matching.

Interpretation

$\pi[q]$ = how much of the pattern can still be reused after failing at position $q + 1$.

4 Worked Example of the Prefix Function

Let

$$P = \text{ababaca}.$$

We compute $\pi[q]$ for each position.

q	$P[q]$	$\pi[q]$
1	a	0
2	b	0
3	a	1
4	b	2
5	a	3
6	c	0
7	a	1

Thus:

$$\pi = [0, 0, 1, 2, 3, 0, 1].$$

Why?

- For $q = 1$: **a** has no proper prefix that is also a suffix, so $\pi[1] = 0$.
- For $q = 3$: **aba** has longest proper prefix-suffix **a**, so $\pi[3] = 1$.
- For $q = 5$: **ababa** has longest proper prefix-suffix **aba**, so $\pi[5] = 3$.
- For $q = 6$: **ababac** has no proper prefix-suffix, so $\pi[6] = 0$.

Important Insight

The prefix function compares the pattern with itself. This self-knowledge is what allows KMP to skip useless shifts.

5 How KMP Uses the Prefix Function

Suppose we have matched

$$P[1 : q]$$

against the text, and then the next character mismatches.

Instead of moving the pattern one step and starting over, KMP moves to:

$$q \leftarrow \pi[q].$$

That means:

use the longest proper prefix of the matched portion that is also a suffix.

So the algorithm preserves as much matching information as possible.

6 The KMP Matcher Algorithm

KMP Matcher

```
KMP-MATCHER(T, P)
1  n = T.length
2  m = P.length
3  pi = COMPUTE-PREFIX-FUNCTION(P)
4  q = 0
5  for i = 1 to n
6      while q > 0 and P[q+1] != T[i]
7          q = pi[q]
8      if P[q+1] == T[i]
9          q = q + 1
10     if q == m
11         print "Pattern occurs with shift", i - m
12         q = pi[q]
```

7 Explanation of the KMP Matcher

Variables

- i : current position in the text,
- q : number of characters of the pattern currently matched,
- π : prefix-function array.

Line-by-Line Idea

- Line 3 computes the prefix function.
- Line 4 starts with no matched characters.
- Lines 6–7 handle mismatch by falling back using $\pi[q]$.
- Lines 8–9 extend the match if the next character agrees.
- Lines 10–11 report a match if the whole pattern has matched.
- Line 12 prepares for the next possible match, allowing overlapping matches.

Why Line 12 is Important

After a complete match, the algorithm does not restart from 0. Instead it continues from:

$$q = \pi[q],$$

so overlapping occurrences are also found.

8 Worked Matching Example

Let

$$P = \text{ababaca}, \quad T = \text{abababacaba}.$$

We already know:

$$\pi = [0, 0, 1, 2, 3, 0, 1].$$

Now track the matching process.

Step-by-step

i	$T[i]$	q after processing	Comment
1	a	1	matched $P[1]$
2	b	2	matched $P[2]$
3	a	3	matched $P[3]$
4	b	4	matched $P[4]$
5	a	5	matched $P[5]$
6	b	4	mismatch with $P[6] = c$, fallback to $\pi[5] = 3$, then match b
7	a	5	matched again
8	c	6	matched $P[6]$
9	a	7	full pattern matched
10	b	2	fallback after full match
11	a	3	continue scanning

At $i = 9$, we have:

$$q = 7 = m.$$

So the pattern occurs with shift:

$$9 - 7 = 2.$$

Result

The pattern **ababaca** occurs in the text **abababacaba** at shift

2.

9 Computing the Prefix Function

The prefix function itself is computed efficiently by another linear-time procedure.

Compute Prefix Function

```
COMPUTE-PREFIX-FUNCTION(P)
1  m = P.length
2  let pi[1:m] be a new array
3  pi[1] = 0
4  k = 0
5  for q = 2 to m
6      while k > 0 and P[k+1] != P[q]
7          k = pi[k]
8      if P[k+1] == P[q]
9          k = k + 1
10     pi[q] = k
11 return pi
```

10 Meaning of the Prefix-Function Algorithm

Here:

- q is the current position whose prefix value we want,
- k stores the current best candidate for $\pi[q - 1]$.

The algorithm extends the previous best border if possible. If not, it repeatedly falls back using the already computed prefix values.

Key Insight

Just as KMP avoids re-comparing text characters, the prefix-function algorithm avoids re-comparing pattern characters unnecessarily.

11 Worked Example: Computing π

Let

$$P = \text{ababaca}.$$

We compute π step by step.

Initialization

$$\pi[1] = 0, \quad k = 0.$$

For $q = 2$

Compare:

$$P[k + 1] = P[1] = a, \quad P[2] = b.$$

Mismatch, so:

$$\pi[2] = 0.$$

For $q = 3$

Compare:

$$P[1] = a, \quad P[3] = a.$$

Match, so:

$$k = 1, \quad \pi[3] = 1.$$

For $q = 4$

Compare:

$$P[2] = b, \quad P[4] = b.$$

Match, so:

$$k = 2, \quad \pi[4] = 2.$$

For $q = 5$

Compare:

$$P[3] = a, \quad P[5] = a.$$

Match, so:

$$k = 3, \quad \pi[5] = 3.$$

For $q = 6$

Compare:

$$P[4] = b, \quad P[6] = c.$$

Mismatch, so fallback:

$$k = \pi[3] = 1.$$

Now compare:

$$P[2] = b, \quad P[6] = c.$$

Mismatch again, fallback:

$$k = \pi[1] = 0.$$

Now compare:

$$P[1] = a, \quad P[6] = c.$$

Still mismatch, so:

$$\pi[6] = 0.$$

For $q = 7$

Compare:

$$P[1] = a, \quad P[7] = a.$$

Match, so:

$$k = 1, \quad \pi[7] = 1.$$

Thus:

$$\pi = [0, 0, 1, 2, 3, 0, 1].$$

12 Running-Time Analysis of Compute-Prefix-Function

At first glance, the while loop seems expensive. But overall, the algorithm is linear.

Why?

- k increases only when a match occurs.
- k decreases only through:

$$k = \pi[k].$$

- k can never become negative.

So across the entire algorithm:

- total increases of k are at most $m - 1$,
- total decreases of k are also at most $m - 1$.

Therefore the total running time is:

$$\Theta(m).$$

Conclusion

The prefix function is computed in linear time.

13 Running-Time Analysis of KMP Matcher

The matcher also runs in linear time.

Why?

As the text is scanned:

- i increases from 1 to n ,
- q increases when characters match,
- q decreases only through prefix-function fallback.

Just as before, total increases and decreases of q are both bounded by n .

So total work is:

$$\Theta(n).$$

Thus the overall KMP complexity is:

$$\Theta(n + m).$$

Why is the matching time linear?

At first sight, the **while** loop in KMP may seem expensive, because for one text position it may execute several times. However, the total number of executions over the whole algorithm is still linear.

The reason is based on the behavior of the variable q :

- q increases only when a character match occurs,
- q decreases only through the fallback step $q = \pi[q]$,
- each fallback strictly decreases q , since $\pi[q] < q$,
- q never becomes negative.

Now, during the entire scan of the text:

- the total number of increases of q is at most n , because each text character can increase q at most once,

- therefore the total number of decreases of q is also at most n .

Hence, although the `while` loop may iterate many times in one step, across the whole execution it performs only $O(n)$ total iterations.

So the matching phase runs in:

$$\boxed{\Theta(n)}.$$

Since computing the prefix function takes $\Theta(m)$, the total running time of KMP is:

$$\boxed{\Theta(n + m)}.$$

14 Prefix-Function Iteration Lemma

A key theoretical result is that repeated application of π lists all proper prefix-suffix lengths.

Lemma 1 (Prefix-Function Iteration Lemma). *For a pattern P , the values*

$$\pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots$$

list all lengths $k < q$ such that P_k is a suffix of P_q .

Meaning

If you start at q and repeatedly follow

$$q \rightarrow \pi[q] \rightarrow \pi[\pi[q]] \rightarrow \dots,$$

you visit all valid smaller borders of P_q .

This explains why fallback using π is enough: no possible useful prefix is missed.

Border Chain

The prefix function creates a chain of fallback states:

$$q, \pi[q], \pi[\pi[q]], \dots, 0.$$

KMP follows this chain until it finds a state from which matching can continue.

15 Proof of the Prefix-Function Iteration Lemma

We now prove why repeated application of the prefix function

$$\pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots$$

lists all proper border lengths of P_q .

Recall the statement

Lemma 2 (Prefix-Function Iteration Lemma). *For a pattern P , the values obtained by repeatedly applying π to q ,*

$$\pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots,$$

are exactly all integers $k < q$ such that P_k is a suffix of P_q .

What we must show

We need to prove two things:

1. every value produced by repeated application of π is a valid border length of P_q ,
2. every valid border length of P_q appears somewhere in this chain.

So the proof has two directions.

Direction 1: Every iterated π -value is a valid border

Let

$$k_1 = \pi[q], \quad k_2 = \pi[k_1], \quad k_3 = \pi[k_2], \quad \dots$$

We want to show that each k_i satisfies:

$$P_{k_i} \text{ is a suffix of } P_q.$$

First step

By definition of the prefix function,

$$\pi[q]$$

is the length of the longest proper prefix of P_q that is also a suffix of P_q .

So immediately,

$$P_{\pi[q]} \text{ is a suffix of } P_q.$$

Thus $k_1 = \pi[q]$ is a valid border length.

Next steps

Now look at

$$k_2 = \pi[k_1].$$

Since $k_1 = \pi[q]$, we know:

$$P_{k_1} \text{ is a suffix of } P_q.$$

Again by definition of π ,

$$P_{k_2} = P_{\pi[k_1]}$$

is a suffix of P_{k_1} .

But if a string is a suffix of P_{k_1} , and P_{k_1} itself is a suffix of P_q , then by transitivity of the suffix relation:

$$P_{k_2} \text{ is also a suffix of } P_q.$$

The same argument works repeatedly: if

$$k_{i+1} = \pi[k_i],$$

then $P_{k_{i+1}}$ is a suffix of P_{k_i} , and since P_{k_i} is already a suffix of P_q , we get

$$P_{k_{i+1}} \text{ is a suffix of } P_q.$$

Hence every value in the chain

$$\pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots$$

is a valid proper border length of P_q .

Conclusion of Direction 1

Every value obtained by repeatedly applying π corresponds to a proper prefix of P_q that is also a suffix of P_q .

Direction 2: Every valid border appears in the chain

Now suppose $j < q$ and

$$P_j \text{ is a suffix of } P_q.$$

We must show that j appears somewhere in the chain

$$\pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots$$

Main idea

The value $\pi[q]$ is the *largest* proper border length of P_q .

So if j is any proper border length of P_q , then either:

- $j = \pi[q]$, in which case we are done, or
- $j < \pi[q]$.

If $j < \pi[q]$, then since $P_{\pi[q]}$ is itself a suffix of P_q , and P_j is also a suffix of P_q , the shorter suffix P_j must also be a suffix of $P_{\pi[q]}$.

So j is a proper border length of $P_{\pi[q]}$.

Then we can apply the same reasoning again:

$$\pi[\pi[q]]$$

is the largest proper border length of $P_{\pi[q]}$.

So either $j = \pi[\pi[q]]$, or j is smaller and remains a border of that smaller prefix too.

Continuing in this way, we keep moving down the chain:

$$q \rightarrow \pi[q] \rightarrow \pi[\pi[q]] \rightarrow \dots$$

Because each application of π strictly decreases the value,

$$\pi[x] < x,$$

this chain must eventually reach 0.

Since j is one of the valid border lengths, it cannot be skipped forever. At some stage it must appear exactly.

Conclusion of Direction 2

Every proper border length of P_q must occur somewhere in the iterated π -chain.

Putting both directions together

From Direction 1:

- every iterated π -value is a valid proper border length.

From Direction 2:

- every valid proper border length appears in the iterated π -chain.

Therefore, the iterated values

$$\pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots$$

are exactly the lengths $k < q$ such that

$$P_k \text{ is a suffix of } P_q.$$

Hence the lemma is proved.

Final Result

Repeated application of the prefix function produces all proper border lengths of P_q , from largest to smallest:

$$q, \pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots, 0.$$

So KMP loses no useful information when it follows fallback links through π .

16 Correctness Idea of KMP

KMP can be viewed as simulating the finite-automaton matcher, but without explicitly storing the full transition table.

Instead of precomputing $\delta(q, a)$ for every state and alphabet character, KMP computes the needed transition on the fly using π .

Core Correctness Idea

After reading $T[1 : i]$, the variable q equals the length of the longest prefix of P that is also a suffix of $T[1 : i]$.

This is the same invariant used by the finite-automaton approach.

So when:

$$q = m,$$

the whole pattern has just matched.

17 Why Line 12 is Necessary

After reporting a match, the algorithm executes:

$$q = \pi[q].$$

This is necessary because:

- the next occurrence may overlap with the current one,
- we should continue from the longest reusable suffix-prefix,
- otherwise we may miss overlapping matches or access $P[m + 1]$.

Example

If

$$P = \text{abab}$$

and the text contains

$$\text{ababab},$$

then after finding the first occurrence, the pattern can continue overlapping.

So:

$$q = \pi[4] = 2$$

lets us continue from the correct state.

18 Comparison with Previous Methods

Method	Preprocessing	Matching
Naive Matcher	0	$\mathcal{O}((n - m + 1)m)$
Rabin–Karp	$\Theta(m)$	average good, worst-case $\mathcal{O}((n - m + 1)m)$
Finite Automaton	$\mathcal{O}(m \Sigma)$ or more	$\Theta(n)$
KMP	$\Theta(m)$	$\Theta(n)$

Why KMP is Special

KMP keeps the linear-time matching performance of finite automata, but reduces preprocessing from $\mathcal{O}(m|\Sigma|)$ to:

$$\Theta(m).$$

19 Advantages of KMP

- It guarantees linear-time matching.
- It uses only the prefix-function array instead of a full transition table.
- It never moves backward in the text.
- It avoids repeated comparisons.
- It is efficient both theoretically and practically.

Main Philosophy

KMP uses the structure of the pattern to decide how far it can safely continue after a mismatch.

20 Final Summary

One-Box Summary

$\pi[q]$ = length of longest proper prefix of $P[1 : q]$ that is also a suffix

KMP uses π to avoid useless shifts after mismatches

COMPUTE-PREFIX-FUNCTION(P) runs in $\Theta(m)$

KMP-MATCHER(T, P) runs in $\Theta(n)$

Total time: $\Theta(n + m)$

KMP simulates finite-automaton matching without storing the full transition table

After a full match, continue with $q = \pi[q]$ to allow overlapping matches